

# Prodapt- Bootcamp – PU Capstone Projects

# What We Have Learned

## Key Skills. Real Tools. End-to-End Applications.

### 1 Python Full Stack Development

- Python Fundamentals & OOP
- Data Structures & File Handling
- Modules, Packages & Virtual Environments
- Error Handling & Logging
- FastAPI (Backend Development)
- REST APIs & CRUD Operations
- Request Validation & Exception Handling
- Database Integration (PostgreSQL / MySQL)
- API Testing (Unit & API Tests)
- Frontend Development (Streamlit / HTML, CSS, JavaScript)
- Role-Based Access & Application Security
- End-to-End Application Development



### 2 Data Engineering

- Data Sources & Ingestion
- Data Cleaning & Transformation
- Data Quality Checks (missing values, duplicates, invalid data)
- ETL / ELT Pipelines
- Incremental ETL Processing
- Reusable ETL Modules
- Scheduling & Batch Processing
- Pipeline Run Logging
- Kafka & Airflow (concepts)
- Data Validation & Error Handling
- Working with Structured & Semi-structured Data (CSV, JSON, etc.)



### 3 Cloud Computing

- Cloud Computing Fundamentals (IaaS, PaaS, SaaS)
- Azure Portal & Core Services
- Azure Storage (Blob, File, Queue)
- Azure Databases (Azure SQL, Cosmos DB)
- Azure Resource Manager
- Identity & Access Management (IAM)
- Application Deployment
- Monitoring & Logging
- Security Best Practices
- CI/CD Basics
- DevOps Concepts
- Docker (concepts)
- MLOps Fundamentals (concepts)



# Capstone Projects – Apply What We Learned

Build end-to-end data applications using Python Full Stack, Data Engineering and Cloud concepts

1

## Retail Sales Analytics Platform



### Problem Statement

Retail businesses need to analyze sales, customer behavior and product performance to make data-driven decisions and improve revenue and customer satisfaction.



### Key Approach

- Build data ingestion and ETL pipelines for sales data
- Analyze sales trends, customer segments and business KPIs
- Develop REST APIs and a simple frontend interface for interactive analysis
- Implement data quality checks, testing and logging
- Deploy the application using cloud services



### Dataset (Example)

Retail Sales Dataset  
Format: CSV



### Key Technologies

- Python, Pandas
- FastAPI (REST APIs)
- PostgreSQL / MySQL
- Streamlit (Frontend)
- Azure (Deployment)

2

## Healthcare Patient Management System



### Problem Statement

Hospitals need to manage patient data, appointments, treatments and outcomes efficiently to improve operational efficiency and quality of care.



### Key Approach

- Ingest and transform patient, appointment and admission data using ETL pipelines
- Build REST APIs with CRUD operations
- Implement data quality checks and validation
- Create reports and dashboards for hospital operations
- Implement role-based access and audit logging
- Deploy the application using cloud services



### Dataset (Example)

Hospital Patient Dataset  
Format: CSV



### Key Technologies

- Python, Pandas
- FastAPI (REST APIs)
- PostgreSQL / MySQL
- Streamlit (Frontend)
- Azure (Deployment)

3

## Banking Loan Risk Analysis Application



### Problem Statement

Banks need to assess loan applications, analyze customer risk and monitor transactions to reduce credit risk and improve decision making.



### Key Approach

- Implement incremental ETL processing for transaction files
- Perform data quality checks and validation
- Build REST APIs with filtering, sorting, pagination and date-range queries
- Implement rule-based loan risk analysis (Low/Medium/High)
- Add role-based access for analyst and administrator
- Create a simple frontend interface for loan analysis
- Implement scheduled pipeline execution, logging and testing
- Deploy the application using cloud services



### Dataset (Example)

Banking Loan Dataset  
Format: CSV



### Key Technologies

- Python, Pandas
- FastAPI (REST APIs)
- PostgreSQL / MySQL
- Streamlit (Frontend)
- Azure (Deployment)

**COMMON  
TECHNIQUES  
ACROSS ALL PROJECTS**



Data Ingestion  
& ETL Processing



Data Cleaning  
& Validation



Relational  
Databases  
(PostgreSQL/MySQL)



REST APIs  
(FastAPI)



Simple  
Frontend Interface  
(Streamlit)



Cloud Deployment  
(Azure)



Access Control  
& Security



Testing,  
Logging &  
Scheduling



Real-world Problems. End-to-End Solutions. Measurable Impact.

Learn

Build

Deploy

Grow



# Telecom Network Incident Response System

An end-to-end Python full-stack application for incident analysis and resolution



## Project Overview

Telecom operators receive a large volume of network logs and customer complaints. The system analyzes incidents, identifies root causes and provides insights to improve service reliability and reduce customer impact.



## Key Questions Addressed

- Which regions or towers have highest incidents?
- What are the common types and root causes?
- Which customers are most affected?
- How can we track and monitor incident resolution and improve response time?



## Requirement 1 – Basic

- ✓ Ingest and **transform** network log data and customer complaint data using ETL
- ✓ Create **relational tables** and perform SQL queries for incident analysis
- ✓ Build REST APIs with CRUD operations
- ✓ Develop a simple front-end interface using Streamlit or Flask
- ✓ Implement basic reporting and visual summaries (e.g., incidents by region, tower, type)
- ✓ Add validation, error handling and basic logging



## Requirement 2 – Advanced

- ✓ Implement **incremental ETL** processing for newly received transaction files
- ✓ Create **reusable ETL modules** for extraction, transformation, validation and loading
- ✓ Implement **data-quality checks** for duplicates, missing values, invalid prices and inconsistent dates
- ✓ Add **parameterized API** filtering, sorting, pagination and date-range queries
- ✓ Implement **role-based access** for analyst and administrator functions
- ✓ **Optimize SQL queries** using indexes, aggregations and appropriate schema design
- ✓ Add **automated tests** for ETL functions, APIs and key business calculations
- ✓ Add **scheduled pipeline execution and pipeline-run logging**
- ✓ Build a simple **front-end interface**



## Solution Architecture (High Level)



### Data Sources

- Network Logs
- Incident Records
- Customer Complaints
- Tower & Region Data
- Equipment Data



### ETL Processing

- Data Extraction
- Data Cleaning & Validation
- Transformation
- Incremental Processing
- Load to Database



### Database (PostgreSQL / MySQL)

- Incident Data
- Network Events
- Customer Data
- Tower / Region Data
- Reference Data



### REST API (Flask / FastAPI)

- CRUD Operations
- Filtering, Sorting, Pagination
- Date-range Queries
- Role-based Access
- API Documentation



### Frontend (Streamlit / Flask)

- Dashboard
- Search & Filters
- Incident Details
- Reports & Downloads
- User Interface



### Scheduler & Monitoring

- Scheduled ETL Pipeline
- Pipeline Run Logs
- Error Notifications
- Automated Tests (PyTest)



## Dataset Details

**Dataset:** Telecom Network Incident Dataset

**Format:** CSV

**Key Fields (Examples):**

- incident\_id, incident\_type, incident\_subtype
- tower\_id, region, city
- customer\_id, complaint\_id
- severity, status, start\_time, end\_time
- resolution\_time, affected\_users
- equipment\_id, root\_cause\_code



## Sample User Query & Output



**User Query:** "Show the top 5 regions with highest number of network incidents in the last 30 days."

**Sample Output (Table):**

| Region    | Total Incidents | Critical | Major | Minor |
|-----------|-----------------|----------|-------|-------|
| Chennai   | 120             | 30       | 55    | 35    |
| Mumbai    | 95              | 20       | 40    | 35    |
| Bangalore | 88              | 18       | 42    | 28    |
| Delhi     | 76              | 15       | 33    | 28    |
| Hyderabad | 64              | 12       | 30    | 22    |



## Business Impact

- ✓ Faster incident detection and resolution
- ✓ Reduced network downtime
- ✓ Improved customer satisfaction
- ✓ Better resource allocation and planning
- ✓ Data-driven network operations
- ✓ Improved SLA compliance
- ✓ Actionable insights for network performance



# Telecom Network Incident Response System – Milestone Plan

An end-to-end Python full-stack application for analyzing network incidents and improving service reliability



Total Duration  
**7 Days**  
(Two Milestones)

## 1 MILESTONE 1 – Core Solution Oct 5 – 7



**OBJECTIVE:** Build the working end-to-end solution with core functionalities to analyze network incidents.



### DATA INGESTION & PREPROCESSING

- Ingest network logs and incident data
- Data cleaning & validation
- Handle missing and inconsistent data
- Store in database



### ETL PROCESSING

- Data transformation
- Data validation rules
- Batch / initial load processing
- Reusable ETL modules



### DATABASE

- PostgreSQL / MySQL
- Relational schema design
- Indexes (basic)
- Load processed data



### API LAYER

- Develop REST APIs
- CRUD operations
- Input validation
- Basic filtering
- API documentation (Swagger)



### FRONTEND (UI)

- Simple dashboard
- View incidents
- Search & filters (basic)
- User interface (Streamlit / Flask)



### DELIVERABLES (Milestone 1)



Processed dataset in database



ETL pipeline code



Working REST APIs



Basic front-end application



API documentation (Swagger)



### DEMO OUTCOME



#### User Query

“Show the top 5 regions with highest number of network incidents in the last 30 days.”



#### System Output

- Top regions with incident count
- Incident details and severity
- Trend analysis (daily/weekly)
- Root cause insights
- Recommended resolution steps

## 2 MILESTONE 2 – Advanced Features & Final Deliverables Oct 8 – 11



**OBJECTIVE:** Enhance the solution with advanced requirements, testing, monitoring and finalize the project.



### INCREMENTAL ETL

- Process newly received files
- Reusable ETL modules
- Incremental loading
- Data-quality validation (duplicates, missing values, invalid data, inconsistent dates)



### ADVANCED APIS

- Parameterized filters
- Sorting & pagination
- Date-range queries
- CRUD operations
- API security & role-based access
- Swagger documentation



### DATABASE OPTIMIZATION

- Query optimization
- Indexes
- Aggregations
- Efficient schema design
- Performance tuning



### SCHEDULING & MONITORING

- Scheduled pipeline execution
- Pipeline-run logging
- Error handling
- Monitoring & alerts



### ENHANCED UI

- Advanced search, filters & pagination
- Role-based access (Analyst / Admin)
- Reports & downloads
- Improved user interface



### DELIVERABLES (Milestone 2)



Complete ETL implementation



Advanced REST APIs (with filters & pagination)



Optimized database (with indexes)



Scheduled pipeline & run logs



Enhanced front-end application



Testing scripts and documentation



### BUSINESS IMPACT

- ✓ Faster incident detection and resolution
- ✓ Reduced network downtime
- ✓ Improved service reliability
- ✓ Better resource allocation and planning
- ✓ Data-driven network operations
- ✓ Actionable insights for network performance



**More reliable networks.  
Faster resolutions.  
Higher service availability.**



### MILESTONE SUMMARY



**Milestone 1**  
Core Solution  
(Oct 5 – 7)



~60%  
Project



**Milestone 2**  
Advanced Features & Final Deliverables  
(Oct 8 – 11)



~40%  
Project



Both milestones together deliver the complete end-to-end solution as defined in the project requirements. **Final submission: October 11.**



Healthcare



Banking



Telecom

# ADD-ON TECHNICAL SKILLS (RECOMMENDED)

These add-on skills will help you build more deployable, scalable and production-ready solutions for the **Healthcare, Banking and Telecom** projects.



## ADD-ON SKILL

## WHY IT HELPS

## WHAT YOU WILL LEARN

1

### Cloud Deployment & Azure

Deploy and manage Python full-stack applications in the cloud.



Helps move the application from local environments to a cloud platform, improving accessibility, scalability, reliability and real-world readiness. Useful for hosting ETL pipelines, APIs and front-end applications.



- Azure App Service (Web App)
- Azure Database (SQL Database)
- Azure Storage (Blob Storage)
- Environment & configuration management
- Deploying Python/Streamlit/Flask applications
- Basic cloud security and access management

2

### CI/CD & DevOps Automation

Automate testing, integration and application deployment for reliable and repeatable releases.



Helps automate testing and deployment, making ETL pipelines, APIs and application releases more reliable, consistent and efficient. Reduces manual effort and ensures faster, error-free deployments.



- Git & GitHub workflows
- CI/CD pipelines (GitHub Actions / Azure DevOps)
- Automated testing in pipelines
- Build and deployment automation
- Release management and version control
- Environment configuration and secrets management

3

### Data Pipeline Orchestration & Monitoring

Build reliable, scheduled and monitorable ETL pipelines for production use.



Strengthens scheduled and incremental ETL processing, provides pipeline monitoring and logging, and improves operational reliability across Healthcare, Banking and Telecom solutions.



- Apache Airflow basics
- Scheduled ETL workflows
- Pipeline dependencies and failure handling
- Pipeline run logging and monitoring
- Alerts and notifications
- Monitoring dashboards



**OUTCOME:** These add-on skills will help you build more deployable, automated and production-oriented solutions for the **Healthcare, Banking and Telecom** projects.



**TIP:** Focus on these skills in parallel with the project implementation to get maximum value.

# Coursera Courses

| Add-On Skill                                        | Why It Helps                                                                                                | Course Name                                                 | Instructor | Provider  |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|------------|-----------|
| <b>Cloud Deployment &amp; Azure</b>                 | Deploy Python full-stack applications to the cloud and improve accessibility, scalability, and reliability. | <b>Deploy a website to Azure with Azure App Service</b>     | Microsoft  | Microsoft |
| <b>CI/CD &amp; DevOps Automation</b>                | Automate testing and deployment and make application and ETL releases more reliable and repeatable.         | <b>Deployment and DevOps</b>                                | Microsoft  | Microsoft |
| <b>Data Pipeline Orchestration &amp; Monitoring</b> | Schedule, monitor, and manage ETL workflows with logging, retries, dependencies, and failure handling.      | <b>Data Engineering Workflow Orchestration with Airflow</b> | Edureka    | Edureka   |



# Project Evaluation Rubric

Healthcare | Banking | Telecom Projects



| No.   | Evaluation Area                                                                     |                              | Criteria                                                                                                                                                                                             | Marks |
|-------|-------------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1     |    | Data Ingestion & ETL         | <ul style="list-style-type: none"><li>File ingestion</li><li>Extraction, transformation and loading</li><li>Reusable ETL modules</li><li>Incremental processing</li></ul>                            | 15    |
| 2     |    | Data Cleaning & Quality      | <ul style="list-style-type: none"><li>Missing values</li><li>Duplicates</li><li>Invalid prices/quantities</li><li>Date validation</li><li>Data-quality checks</li></ul>                              | 10    |
| 3     |    | Database Design & SQL        | <ul style="list-style-type: none"><li>Relational schema</li><li>Normalization</li><li>Primary and foreign keys</li><li>Loading data</li><li>KPI queries</li><li>Joins</li><li>Aggregations</li></ul> | 15    |
| 4     |    | Python Implementation        | <ul style="list-style-type: none"><li>Pandas implementation</li><li>Modular code</li><li>Functions/classes</li><li>Exception handling</li><li>Logging</li><li>Code quality</li></ul>                 | 10    |
| 5     |    | REST API & CRUD              | <ul style="list-style-type: none"><li>Flask/FastAPI endpoints</li><li>CRUD operations</li><li>Validation</li><li>Filtering</li><li>Sorting</li><li>Pagination</li><li>Date-range queries</li></ul>   | 15    |
| 6     |    | Analytics & Business KPIs    | <ul style="list-style-type: none"><li>Revenue</li><li>Quantity sold</li><li>AOV</li><li>Product/category performance</li><li>Regional/monthly analysis</li><li>Declining products</li></ul>          | 10    |
| 7     |    | Dashboard / Front End        | <ul style="list-style-type: none"><li>Interactive UI</li><li>KPI cards</li><li>Charts</li><li>Filters</li><li>Usability</li><li>Integration with APIs/database</li></ul>                             | 10    |
| 8     |   | Testing & Reliability        | <ul style="list-style-type: none"><li>Unit tests for ETL, APIs and calculations</li><li>Error handling</li><li>Validation</li><li>Pipeline logging</li></ul>                                         | 5     |
| 9     |  | Architecture & Design        | <ul style="list-style-type: none"><li>Architecture diagram</li><li>Design decisions</li><li>Modularity</li><li>Scalability</li><li>Trade-offs</li></ul>                                              | 5     |
| 10    |  | Documentation & Presentation | <ul style="list-style-type: none"><li>README</li><li>Setup instructions</li><li>Database/ETL documentation</li><li>Demo</li><li>Explanation and Q&amp;A</li></ul>                                    | 5     |
| Total |                                                                                     |                              |                                                                                                                                                                                                      | 100   |



# PARTICIPANT EXPECTATIONS & SUBMISSION GUIDELINES

Python Full-Stack Project – Healthcare | Banking | Telecom



Healthcare



Banking



Telecom



## EXPECTATIONS FROM PARTICIPANTS



### Understand the Problem

Read the problem statement and business requirements thoroughly and identify key use cases.



### Apply Learned Skills

Use Python, SQL, ETL, APIs, database, and front-end skills to build a complete end-to-end solution.



### Work with Realistic Data

Ingest, clean and process the provided datasets. Handle missing values, invalid data and inconsistencies.



### Write Clean & Modular Code

Follow best coding practices. Use functions/classes, proper file structure, logging and exception handling.



### Document Your Work

Provide clear documentation for setup, usage, APIs, database design and design decisions.



### Test and Validate

Test ETL pipelines, APIs and business calculations. Handle errors and edge cases.



### Work Ethically & Collaboratively

Original work only. Acknowledge any external references. Collaborate effectively within your team.



### Present with Clarity

Explain your approach, design and results during the demo and Q&A session.



## SUBMISSION PROCESS

1



### Step 1 Prepare Project Files

Complete all required files as per the structure and checklist.

2



### Step 2 Compress as ZIP

Compress the complete project folder into a single ZIP file.

3



### Step 3 Upload to GitHub

Upload the ZIP file in your team's GitHub repository.

4



### Step 4 Submit Details

Share the GitHub URL and other details as per the submission format.

5



### Step 5 Confirmation

Ensure all files are uploaded successfully and accessible. Verify with the organizers.



### Submit:

Share your GitHub Repository URL with the organizers along with the following:

- GitHub Repository URL
- Demo Video URL (Optional)
- Team Details (Names, ID, Email)

## REQUIRED SUBMISSION FILES & CONTENT



### 1. Architecture Diagram (PDF/JPEG)

- High-level system architecture
- Data flow
- Components (ETL, Database, APIs, UI)

File Name:  
01\_Architecture.pdf



### 2. Design Document (PDF)

- Design decisions
- Technology choices
- Trade-offs
- Scalability considerations

File Name:  
02\_Design\_Document.pdf



### 3. Source Code (Folder/ZIP)

- Complete source code
- Well-structured and modular
- Include all dependencies
- Follow folder structure

Folder Name:  
03\_Source\_Code/



### 4. README File (Markdown/PDF)

- Project overview
- Setup instructions
- How to run
- API endpoints
- Sample queries & outputs

File Name:  
04\_README.md



### 5. API Documentation (PDF/Markdown)

- List of all APIs
- Request format
- Response format
- Example requests & responses

File Name:  
05\_API\_Docs.pdf



### 6. Dataset Details (PDF/CSV)

- Dataset name & description
- Source (if any)
- Key fields description
- Sample data

File Name:  
06\_Dataset\_Details.pdf



### 7. Demonstration Video (MP4)

- Working demo of the solution
- Max 5–7 minutes
- Upload link (Google Drive / YouTube – Unlisted)

File Name:  
07\_Demo\_Video.mp4



### 8. Presentation Deck (MPPT/PDF)

- 10-minute presentation
- Problem, approach, demo, results, impact

File Name:  
08\_Presentation.pdf



## RECOMMENDED FOLDER STRUCTURE (INSIDE SOURCE CODE FOLDER)



/app  
Main application code



/data  
Datasets (raw and processed)



/etl  
ETL scripts



/db  
Database scripts and queries



/api  
API code



/frontend  
UI code



/tests  
Test cases and sample outputs



/docs  
Documentation



/utils  
Utility functions



## SUBMISSION CHECKLIST (BEFORE UPLOADING)

- ✓ All required files are included.
- ✓ Code runs successfully as per README.
- ✓ APIs are tested and working.
- ✓ Architecture & design are clear and complete.
- ✓ Demo video and presentation are included.
- ✓ Files are zipped and named correctly.
- ✓ GitHub repository is accessible.
- ✓ Team details are provided.

Q & A

